

They might require storing a substantial amount of data cluster-wide which decreases scalability headroom.

- They might require implementation of non-trivial data aggregation that reduces resilience to single node failure.
- They are harder to build due to the need of being able to run mixed deployments. Cluster-wide features need to take this into account.
- They might affect our ability to provide an on-premise like experience on GitLab.com.
- Some features that are expected to be cluster-wide might in fact be better implemented using aggregation techniques that use trusted intra-cluster communication using the same user identity.

For example, user Profile is shared across the cluster.

- The Cells architecture limits what services can be considered cluster-wide.

Services that might initially be cluster-wide are still expected to be split in the future to achieve full service isolation.

No feature should be built to depend on such a service (like Elasticsearch).

Will Cells use the reference architecture for up to 1000 RPS or 50,000 users?

See reference architecture for up to 1000 RPS or 50,000 users.

The infrastructure team will properly size Cells depending on the load.

The Tenant Scale team sees an opportunity to use GitLab Dedicated as a base for Cells deployment.

Decision log

- ADR-001: Routing Technology using Cloudflare Workers
- ADR-002: One GCP Project per Cell
- ADR-003: One GKE Cluster per Cell
- ADR-004: One VPC per Cell, with Private Service Connect for internal communication between Cells
- ADR-005: Cells use Flexible Reference Architectures
- ADR-006: Use Geo for Disaster Recovery
- ADR-007: Cells 1.0 for internal customers only
- ADR-008: Cluster wide unique database sequences
- ADR-009: Initial Cell Sizes
- ADR-010: HTTP Router uses static rules and HTTP-based caching mechanism
- ADR-011: Cell Specific Configuration
- ADR-012: Cell Unique Identifier
- ADR 013: Use the same Cell ID for restoring a Cell from backup
- ADR 014: No clusterwide syncing in Cells 1.0

Links

- Internal Pods presentation
- Cells Epic
- Database group investigation
- Shopify Pods architecture
- Opstrace architecture
- Adding Diagrams to this blueprint

SECTION: engineering/architecture/design-documents/cells/feature_flags.md

{{< engineering/design-document-header >}}

Summary

This document is to cover GitLab current usage and future use of Feature Flags when used in Development specifically with use on the Cells Infrastructure. Reference our development documentation for Feature Flags here:

- Development Documentation
- Blueprint Implementation
- Blueprint Operations Proposal

:warning: Do not confuse this with the GitLab Application feature set of Feature Flags.

Current Usage

Feature Flags play a vital role in the Development of GitLab. Chunks of code have the ability to be tested in a safe manner increasing confidence that a new feature will operate both as desired and at the scale of .com. Cells introduces a new set of challenges on top of already existing bits of technical debt related to the wide adoption of Feature Flags. Reference our existing blueprint for Feature Flags for further information: Feature Flag Use Cases

Challenges in Cells

- Discovery capabilities to understand what actors exist on which Cells
- Rollout capabilities per Cell do not exist
- Management of feature flag state is not centralized
- Expansion of Cells brings burden to Development and Operations teams

Proposal

We will iterate on expanding our use of Feature Flags into Cells, but far slower as the Legacy Cell, or the current .com Infrastructure, will still be the primary place that we can expect the majority of feature flags to be leveraged for testing. Because the migration of customers onto Cells is a goal in the future we'll certainly want to expand how we interact with Feature Flags for target actors, but we need to develop the capabilities and procedures to ensure the safety and stability of .com.

Iteration Cells 1.0

With the ultimate goal that Cells is attempting to host more stable versions of GitLab, our first Iteration for feature flags will mostly be discovery of work to be accomplished along with prioritizing with the appropriate teams. Doing so also provides us time to think about future iterations and refine as we see fit.

The expectation here is that we'll continue to use our Legacy Cell, our current infrastructure,

to manage and change feature flags like we do today with no change in the behavior or the expectations by Development teams related to any Cell. Any on-going work surrounding improvements that may already be in progress should be accounted for as to prevent interference or introduce unnecessary complexity.

Future Iterations

Adding Capabilities

Engagement on Cells

Historically, Feature Flags have the potential to mitigate incidents. For example, if a feature is behind a feature flag and enabled, but the code may not be behaving as desired, we can use this as leverage to mitigate incidents. In another example, a feature may be under heavy development and we need to gather additional information prior to enabling that feature Cluster wide. We could provide a mechanism where development teams directly engage with a Cell. In order to accomplish this, we'll need to expand the capabilities of Chatops to understand the concept of Cells as well as the ability to provide a UX for which Engineering teams can execute a command to target a Cell.

Let's use an example. Let's say we want to change the feature flag loremipsumdolar on Cell 7 because of an identified incident related to the code sitting behind this flag. Using the command:

```
> /chatops run feature set loremipsumdolar false --cell 7
```

This will reach out to Cell 7 and disable the feature flag.

Engagement on Actors

Feature Flags that are leveraged to gather information to assist with development may target a specific project, user, or percentage based actor. Issues exist which that make setting this across all Cells difficult. Thus, for actor based changes to feature flags, these will be limited to only the Legacy Cell.

Let's use an example. Let's say we want to change feature flag loremipsumdolar for actor @ayufan. This user may be spread across 3 total Cells. Using the command:

```
> /chatops run feature set loremipsumdolar ayufan
```

The command only change the flag for that actor on our Legacy Cell. All other cells will be ignored. We may be able to expand Chatops to be able to accept an added flag such that we could directly set the actor on a particular Cell. Doing so will require the Engineer to know which Cell an actor resides. The reason for these limitations is to account for the fact that users and projects may be spread across a multitude of Cells. Cells are also being designed such that we can migrate data from one Cell to another. Feature Flag data is stored as a setting on a Cell and thus the metadata associated with what flags are set are not part of the knowledge associated with an actor. This introduces risk that if we target an actor, and later that actor is moved, the flag would no longer be set properly.

This will lead to differing behavior for a given actor, but normally these types of changes happen to internal customers to GitLab reducing risk that users will notice a behavioral change as they switch between Cells. This implementation is also simplistic, removing the need to query some service which hosts the Cells and actor resides, and needing to develop a specialized rollout procedure when the resulting target may be more than a single Cell. This is discussed a bit more in the next section.

Engagement on Environments

Today we have the ability to set a feature flag for an entire environment. Production is one of them. This begs the question, how do we roll out feature flags to all Cells? Ideally the flag will have been well tested, but we may still need some sort of testing to validate behavior, prior to rolling the flag to all Cells.

Let's use an example. Let's say we want to enable the flag `loremipsumdolar` on all of Production. Using the command:

```
> /chatops run feature set loremipsumdolar true --production
```

This command will need to perform a lot of work. Firstly, it needs to gather all Cells where this feature flag exists. If the flag does not exist on any Cell, we must not change this as this introduces an consistency issue between the Engineer expectations and that of the Production environment. We may consider an override in the case we are attempting to leverage this to mitigate an incident of a deployment that is not fully completed, however. If the flag does exist across all Cells of a given environment, we then begin to roll that change out across all Cells. It would be inadvisable to change all Cells at the same time. Chatops now needs the ability to have some mechanism to make the change to a given list of Cells, wait for some signal, then proceeding to the next list of Cells. Repeating until completion. We may need a mechanism to bypass this intentionally built slow rollout if we are targeting a flag that may remediate an incident across all Cells. The Delivery team plan on using a Ring style of deployments for Cells, we may be able to leverage similar metadata to assist in rollouts for this use case.

Requirements

- Chatops needs the ability to talk to some service to gather requisite information. This may include:
 - The listing of Cells
 - Actors assigned to a listing of Cells
- When a new Cell is brought online, we need not manage configuration for Chatops. Instead, Chatops should have automated access to new Cells. This minimizes the administrative burden whenever a Cell is newly constructed.
- Procedure refinement. We already have existing efforts to improve various aspects of Feature Flags today. We should remain cognizant of this on-going work.

SECTION: engineering/architecture/design-documents/cells/goals.md

Goals

Scalability

The main goal of this new shared-infrastructure architecture is to provide additional scalability for our SaaS Platform.

GitLab.com is largely monolithic and we have estimated (internally) that the current architecture has scalability limitations, particularly for the PostgreSQL database, and Redis non-horizontally scalable resources, even when database partitioning and decomposition are taken into account.

Cells provide a horizontally scalable solution because additional Cells can be created based on demand. Cells can be provisioned and tuned as needed for optimal scalability.

Increased availability

A major challenge for shared-infrastructure architectures is a lack of isolation between top-level Groups.

This can lead to noisy neighbor effects.

An organization's behavior inside a top-level Group can impact all other organizations.

This is highly undesirable.

Cells provide isolation at the Cell level.

A group of Organizations is fully isolated from other Organizations located on a different Cell.

This minimizes noisy neighbor effects while still benefiting from the cost-efficiency of a shared infrastructure.

Additionally, Cells provide a way to implement disaster recovery capabilities.

Entire Cells may be replicated to read-only standbys with automatic failover capabilities.

A consistent experience

Organizations should have the same user experience on our SaaS platform as they do on a self-managed GitLab instance.

Regions

GitLab.com is only hosted within the United States of America.

Organizations located in other regions have voiced demand for local SaaS offerings.

Cells provide a path towards GitLab Regions because Cells may be deployed within different geographies.

Depending on which of an organization's data is located outside a Cell, this may solve data residency and compliance problems.

Market segment

At this moment, GitLab.com has "social network"-like capabilities that may not fit well into a more isolated Organization model.

Removing those features, however, poses some challenges:

1. How will existing gitlab-org contributors contribute to the namespace?

1. How do we move existing top-level Groups into the new model (effectively breaking their social features)?

We should evaluate if the small to medium business and mid-market segment is interested in these features, or if not having them is acceptable in most cases.

Self-managed

For reasons of consistency, it is expected that self-managed instances will adopt the single cell architecture as well. Self-managed instances can continue with just a single Cell without needing the additional Cell services like routing or topology. Organizations, and possible User decomposition will also be adopted for self-managed instances.

Requirements

Type	Requirement	Severity
Product	Aggregation of cluster-wide data	Medium
Product	All Cells are under a single GitLab.com domain	High
Product	User can interact with many Cells	Medium
Product	Minimal impact on contributor workflows	High
Product	On-premise like experience	Medium
Product	Minimize breaking changes	High
Product	Enables support for Regions	Low
Product	Limit impact on self-managed customers	Low
Operational	Provides 10x headroom	High
Operational	Provides 100x headroom	Medium
Operational	Improve Service Availability	High
Operational	Cost per user similar or lower to GitLab.com	Medium
Operational	Unified way of deploying Cells	Medium
Operational	Cells running in mixed deployments	High
Operational	High resilience to a single Cell failure	High
Operational	Small Cells	Medium
Migration	Robust rollback and disaster recovery scenarios	High
Migration	Scale existing GitLab.com database	High
Migration	Re-balancing of Cells	Low
Migration	Customer can migrate from GitLab.com to Dedicated	Low
Development	Easy usage in development environment	Medium

Aggregation of cluster-wide data

The architecture should provide a way to present the cluster of Cells in a single view. This might mean to:

- Aggregate user To-Dos from different Organizations
- Perform cluster-wide searches of public Projects, or published source code

All Cells are under a single GitLab.com domain

General users should be unaware of the presence of Cells.
Cells would only be visible to instance administrators.
Users using Organizations on another Cell or us migrating Organizations between Cells should be an operational detail that does not require any user intervention or is not visible to the user.

User can interact with many Cells

Usage of many accounts to interact with different Organizations that might be on different Cells is strongly discouraged.
The need to use many accounts will reduce adoption and put a barrier on users wanting to contribute to open-source Projects.

In the future, users could have Organization specific settings, depending on the specific customer expectations.
For instance, users might be shown with different display names in different Organizations.

Users should not be required to use different SSH keys to access different Organizations due to the complexity of managing many SSH keys by the user.

Minimal impact on contributor workflows

GitLab.com has a number of open-source and open-core projects (including gitlab-org/gitlab).
Cells should not make it harder to contribute to public Projects.
Learning new ways to contribute is likely to hinder the adoption of Cells.
The introduced architecture should focus on changing the existing workflows in the most minimal way.

On-premise like experience

Currently on-premise has numerous advantages compared to our SaaS offering.
On-premise allows to control all aspects of a GitLab installation, including but not limited to: managing users, access controls, or instance-wide settings.

The difference between SaaS and on-premise is problematic for our customers, as it results in a different user experience.

Minimize breaking changes

The introduction of Cells will imply changes in the supported GitLab.com workflows.
Cells might affect how people use GitLab, as long as it provides a compelling user experience story.
It is desired that each change introduced provides either operational value or user value, ideally both.

Any Cells architecture should focus on not introducing breaking changes, or if introducing them provide a long window to support the change.
The introduction of breaking changes will reduce the adoption rate and effectiveness of Cells and make rollout of the architecture more risky.

It is important to distinguish between different severity of breaking changes:

- Required: migration to a new architecture by design introduces a breaking change.
In such a case a customer has to use a new workflow, new API, or new endpoints right away and is required to adapt to all changes at once. The customer has limited options to roll back.
- Optional: migration to a new architecture does not enforce introducing breaking changes.
In such a case a customer can slowly adapt to introduced changes: a new workflow, new API, or new endpoints, but retaining major aspects of the system to work as before.
The customer has a way to roll back to a previous known working state.

Enables support for Regions

Support for Regions should enable us to run GitLab in different availability zones, or completely different data centers.

This includes, but is not limited to allowing users to create Organizations in Europe, the US West or US East, and GitLab Inc. to use different cloud infrastructure providers (Google Cloud, AWS) to serve customers.

Cells enable existing customers running Organizations to move their Organization between different deployment regions/data centers that are supported by GitLab on GitLab.com.

Limit impact on self-managed customers (Omnibus/CNG)

The introduction of Cells should not impact small installations.
Cells should not require running additional components by self-managed customers unless they are interested in Cells that will increase resource requirements.

Provides 10x headroom

The Cells architecture must provide at least 10x headroom. As such our architecture must be suitable to run 10 Cells. We do not need to initially solve for the complexities that might come with running more than 10 Cells.

Provides 100x headroom

The Cells architecture should be able to run more than 10 Cells.

Improve Service Availability

The Cells architecture should allow us to provide better SLA for some customers by putting them on specific Cells of the cluster:

- Cells have to reduce the impact of noisy neighbors, such as legitimate usage caused by a spiky workload.
- Cells have to reduce the impact caused by abuse, for instance CI being used for cryptocurrency mining.

Cost per user similar or lower to GitLab.com

The current GitLab.com architecture is rather cost-effective.

The introduction of Cells will result in more infrastructure components due to data distribution:

- The proposed Cells architecture should evaluate the cost of running additional Cells with respect to the achieved infrastructure component isolation. In some cases it might be beneficial to reuse infrastructure components to reduce the running cost of Cells.
- The proposed Cells architecture should ensure a high degree of multi-tenancy on Cells.
- The proposed Cells architecture could enable a way to balance the Cells load long term.

Unified way of deploying Cells

The proposed Cells architecture anticipates a need to run 100 Cells and more in a cluster. This becomes operational overhead. It is strongly desired for the Cells architecture to reuse existing infrastructure tooling as much as possible for: deployment, monitoring, logging, disaster recovery, and customer support.

Cells running in mixed deployments

It is strongly required for the Cells architecture to run different versions of applications across the cluster. The purpose is to provide staged deployments allowing to test changes on part of the cluster, and to update part of the cluster less frequently.

This also indicates that Cells should be able to work with different versions of the underlying infrastructure components: PostgreSQL version, Redis version, Gitaly version, Elasticsearch version.

Usage of different versions within a Cell has to have no effect on other Cells.

High resilience to a single Cell failure

Unavailability of a single Cell should not result in other Cells being unable to function properly:

- Intra-cluster communication between Cells should be resilient to single Cell failure.
- Intra-cluster communication should limit its dependence on data stored in other Cells.
- Cluster-shared data should provide an anti-corruption layer to prevent a single Cell from causing a cluster-wide outage.
- Cells should be monitored and their availability status should be accessible to all other Cells.
- Cluster-wide data aggregation should take the availability status of each Cell into account.
- The loss of cell-local data (groups and projects) does only affect ability to access data located on this cell.

Small Cells

The Cells architecture should provide a cost effective way to run small Cells. Running small Cells allows to achieve superior quality in terms of latency or performance due to processing a significantly smaller data set and allowing significant single-node vertical scaling.

The ability to run small Cells could result in reduced complexity of individual Cell deployment:

- Prefer single-node vertical scaling as a first technique for increasing capacity (increase CPU count, available memory).
- Reduce amount of data stored within a Cell to reduce time-to-recovery in case of outages.
- Less data results in faster database migrations, improved latency, and superior user-facing performance.

Robust rollback and disaster recovery scenarios

Rollout of the Cells architecture is a fundamental change to how GitLab.com will operate. This will involve introducing a number of new components, and distributing data horizontally across different Cells.

We anticipate that something will go wrong at some point. Each deployment phase of the Cells architecture needs to provide a way to roll back to a prior working state. This includes any user-facing changes, as well as any infrastructure deployed components.

Scale existing GitLab.com database

The existing GitLab.com database will continue to grow while we deploy Cells. The Cells architecture should provide a way to increase Cell 1 (existing GitLab.com) capacity as soon as possible.

Re-balancing of Cells

It is expected that data distribution on Cells will become uneven at some point.

The problem of re-balancing Cells is rather complex, but at some scale it might provide a positive ROI compared to the running cost of imbalanced Cells.

This might involve understanding how to selectively migrate customer data between Cells onto already existing Cells.

Customer can migrate from GitLab.com to Dedicated

Cells will enforce an architecture with more isolation. However, it is expected that some customers would like to move from SaaS GitLab.com to GitLab Dedicated.

This implies being able to selectively migrate data from the GitLab.com cluster to Dedicated. This could be achieved by:

- Migrating a customer to a Dedicated Cell on the GitLab.com cluster.

- Disconnecting a Cell from the GitLab.com cluster into its own standalone Dedicated Cell.

Easy usage in development environment

The Cells architecture should be easy to run locally by developers as needed to be able to:

- Implement cluster-wide features.
- Model Cells not being available.

Non-Goals

The following objectives are not part of this document. At some point they were considered, but dismissed as creating distraction from the goals and requirements documented above.

Federation between distinct GitLab instances

The Cells architecture is meant to provide trusted intra-cluster communication with some set of data being shared. Federation is meant to solve the problem of data flow between two completely distinct and external instances.

Merging Dedicated instance(s) to GitLab.com

It is not intended for any existing Dedicated instances to join the GitLab.com cluster.

Usage of cloud managed services

Cells should not interfere with other efforts to move towards using more managed services in the cloud.

Cells should replace canary deployments

When we have Cells we won't need to run canary deployments anymore as we can roll out changes to single Cells at a time.

Our main benefit of canary deployments today is that we can roll out code changes to internal users first.

With Cells, we will have internal users on a specific Cell, which can be deployed first.

Additionally Cells may solve some issues with canaries today, for example not having a way to upgrade Sidekiq code for a subset of users.

Glossary

Term	Description
Discouraged Terms	
-----	-----

Cell	A Cell is a set of infrastructure components that contains

multiple top-level Groups that belong to different Organizations. | GitLab
instance, cluster, shard, Pod |
| Legacy Cell | The existing GitLab.com infrastructure
| Primary Cell |
| Cluster | A cluster is a collection of Cells.
| Whale, GitLab Dedicated instance, instance |
| Organizations | An Organization is the umbrella for one or multiple top-level Groups.
| Billable entities, customers |
| Top-Level Group | Top-level Group is the name given to the top-most Group of all other Groups. Groups and Projects are nested underneath the top-level Group. | Root-level namespace
|
| Users | An account containing its own personal namespace associated with an email. | Customer
|

Cell

> - Pod was renamed to Cell in <<https://gitlab.com/gitlab-com/www-gitlab-com/-/mergerequests/121163>>

A Cell is a set of infrastructure components that contains multiple top-level Groups that belong to different Organizations. The components include both data stores (PostgreSQL, Redis, etc.) and stateless services (web, etc.). The infrastructure components provided within a Cell are shared among Organizations and their top-level Groups, but are not shared with other Cells. This isolation of infrastructure components means that Cells are independent from each other.

- Each Cell is independent from other Cells
- Infrastructure components are shared by Organizations and their top-level Groups within a Cell
- More Cells can be provisioned to provide horizontal scalability
- A failing Cell does not lead to failure of other Cells
- Noisy neighbor effects are limited to within a Cell
- Cells are not visible to Organizations - they are an implementation detail
- Cells may be located in different geographical regions (for example: EU, US, JP, UK)
- On initial provisioning, a GitLab instance provisioned by GitLab Dedicated tooling can be configured to join the cluster, and become a Cell

Discouraged synonyms: GitLab instance, cluster, shard, Pod

Legacy Cell

The existing GitLab.com infrastructure will operate as a Cell from an application perspective. We refer to this as legacy because it's where our existing customers live, and it's using the old provisioning tooling (not GitLab Dedicated tooling). Eventually, we also want to migrate customers from the Legacy Cell to the Cells we provision.

Cluster

A cluster is a collection of Cells.

Discouraged synonyms: Whale, GitLab Dedicated instance, instance

Organizations

An Organization is the umbrella for one or multiple top-level Groups. Organizations are isolated from each other by default meaning that cross-namespace features will only work for namespaces that exist in a single Organization.

See the Organization blueprint.

Organizations are a known concept, present for example in AWS and GCP.

Organizations work under the following assumptions:

1. Users care about what happens within their Organization.
1. Features need to work within an Organization.
1. Only few features need to work across Organizations.
1. Users understand that the majority of pages they view are only scoped to a single Organization at a time.
1. Organizations are located on a single Cell. An organization will not span across Cells.
1. A Cell will be multi-tenant. For efficiency, and cost reasons, a Cell will accommodate multiple Organizations.
1. An Organization is the intended top-level entity for a customer. We will not build features to support aggregation of functionality across multiple Organizations for a single customer. In other words, only one Organization is provisioned for one customer.

Properties:

- Top-level Groups belong to Organizations.
- Organizations are isolated from each other by default, meaning that cross-Group features will only work for Groups that exist within a single Organization.
- Personal namespaces must not belong to an Organization.

Discouraged synonyms: Billable entities, customers

Top-level Group

Example:

<https://gitlab.com/gitlab-org/gitlab/>:

- gitlab-org is a top-level group; the root for all Groups and Projects of an Organization
- gitlab is a project; a Project of the Organization.

The top-level Group has served as the de facto Organization entity. With the creation of Organizations, top-level Groups will be nested underneath Organizations.

Over time, there won't be a distinction between a top-level Group and a Group. All features that make top-level Groups different from Groups will move to the Organization.

- Top-level Groups belonging to the same Organization are located on the same Cell.
- Top-level Groups can interact with other top-level Groups that belong to the same Organization.

Discouraged synonyms: Root-level namespace

Users

Users are available globally and not restricted to a single Cell. Users belong to a single Organization, but can participate in many Organizations through Group and Project membership with varying permissions. Inside Organizations, users can create multiple top-level Groups. User activity is not limited to a single Organization but their contributions (for example To-Dos) are only aggregated within an Organization. This avoids the need for aggregating across Cells.

- Users are shared globally across all Cells.
- Users can create multiple top-level Groups.
- Users can be members of multiple top-level Groups.
- Users belong to one Organization. See !395736.
- Users can be members of Groups and Projects in different Organizations.
- Users can administer Organizations.
- User activity is aggregated in an Organization.
- Every user has one personal namespace.

SECTION: engineering/architecture/design-documents/cells/http_routing_service.md

This document describes design goals and architecture of Routing Service used by Cells. To better understand where the Routing Service fits into architecture take a look at Infrastructure Architecture.

Goals

The routing layer is meant to offer a consistent user experience where all Cells are presented under a single domain (for example, gitlab.com), instead of having to go to separate domains.

The user will be able to use <https://gitlab.com> to access Cell-enabled GitLab. Depending on the URL access, it will be transparently proxied to the correct Cell that can serve this particular information.

For example:

- All requests going to <https://gitlab.com/users/signin> are randomly distributed to all Cells.
- All requests going to <https://gitlab.com/gitlab-org/gitlab/-/tree/master> are always directed to Cell 5, for example.
- All requests going to <https://gitlab.com/my-username/my-project> are always directed to Cell 1.

1. Technology.

We decide what technology the routing service is written in.

The choice is dependent on the best performing language, and the expected way and place of deployment of the routing layer.

If it is required to make the service multi-cloud it might be required to deploy it to the CDN provider.

Then the service needs to be written using a technology compatible with the CDN provider.

ADR 001

1. Cell discovery.

The routing service needs to be able to discover and monitor the health of all Cells.

1. User can use single domain to interact with many Cells.

The routing service will intelligently route all requests to Cells based on the resource being

accessed versus the Cell containing the data.

1. Router endpoints classification.

The stateless routing service will fetch and cache information about endpoints from one of the Cells.

We need to implement a protocol that will allow us to accurately describe the incoming request (its fingerprint), so it can be classified by one of the Cells, and the results of that can be cached.

We also need to implement a mechanism for negative cache and cache eviction.

1. GraphQL and other ambiguous endpoints.

Most endpoints have a unique classification key: the Organization, which directly or indirectly (via a Group or Project) can be used to classify endpoints.

Some endpoints are ambiguous in their usage (they don't encode the classification key), or the classification key is stored deep in the payload.

In these cases, we need to decide how to handle endpoints like `/api/graphql`.

1. Small.

The Routing Service is configuration-driven and rules-driven, and does not implement any business logic.

The maximum size of the project source code in initial phase is 1000 lines without tests.

The reason for the hard limit is to make the Routing Service to not have any special logic,

and could be rewritten into any technology in a matter of a few days.

Requirements

Requirement Priority	Description	
-----	-----	
Discovery high	needs to be able to discover and monitor the health of all Cells.	
Security high	only authorized cells can be routed to	
Single domain high	for example GitLab.com	
Caching high	can cache routing information for performance	
Low latency high	50 ms of increased latency	high
Path-based high	can make routing decision based on path	
Complexity high	the routing service should be configuration-driven and small	
Rolling high	the routing service works with Cells running mixed versions	
Feature Flags high	features can be turned on, off, and % rollout	
Progressive Rollout medium	we can slowly rollout a change	
Stateless medium	does not need database, Cells provide all routing information	
Secrets-based medium	can make routing decision based on secret (for example JWT)	
Observability 	can use existing observability tooling	low
Self-managed 	can be eventually used by self-managed	low
Regional 	can route requests to different regions	low

Low Latency

The target latency for routing service should be less than 50 ms.

Looking at the urgency: high request we don't have a lot of headroom on the p50.
Adding an extra 50 ms allows us to still be in or SLO on the p95 level.

There is 3 primary entry points for the application; web, api, and git.

Each service is assigned a Service Level Indicator (SLI) based on latency using the apdex standard.

The corresponding Service Level Objectives (SLOs) for these SLIs require low latencies for large amount of requests.

It's crucial to ensure that the addition of the routing layer in front of these services does not impact the SLIs.

The routing layer is a proxy for these services, and we lack a comprehensive SLI monitoring system for the entire request flow (including components like the Edge network and Load Balancers) we use the SLIs for web, git, and api as a target.

The main SLI we use is the rails requests.

It has multiple satisfied targets (apdex) depending on the request urgency:

Urgency	Duration in ms
high	250 ms
medium	500 ms
default	1000 ms
low	5000 ms

Analysis

The way we calculate the headroom we have is by using the following:

math

$$\mathrm{Headroom} \{ms\} = \mathrm{Satisfied} \{ms\} - \mathrm{Duration} \{ms\}$$

web:

Target Duration	Percentile	Headroom
5000 ms	p99	4000 ms
5000 ms	p95	4500 ms
5000 ms	p90	4600 ms
5000 ms	p50	4900 ms
1000 ms	p99	500 ms
1000 ms	p95	740 ms
1000 ms	p90	840 ms
1000 ms	p50	900 ms
500 ms	p99	0 ms
500 ms	p95	60 ms
500 ms	p90	100 ms
500 ms	p50	400 ms
250 ms	p99	140 ms
250 ms	p95	170 ms
250 ms	p90	180 ms
250 ms	p50	200 ms

Analysis was done in <<https://gitlab.com/gitlab-org/gitlab/-/issues/432934note1667993089>>

api:

Target Duration	Percentile	Headroom
5000 ms	p99	3500 ms

5000 ms	p95	4300 ms
5000 ms	p90	4600 ms
5000 ms	p50	4900 ms
1000 ms	p99	440 ms
1000 ms	p95	750 ms
1000 ms	p90	830 ms
1000 ms	p50	950 ms
500 ms	p99	450 ms
500 ms	p95	480 ms
500 ms	p90	490 ms
500 ms	p50	490 ms
250 ms	p99	90 ms
250 ms	p95	170 ms
250 ms	p90	210 ms
250 ms	p50	230 ms

Analysis was done in <<https://gitlab.com/gitlab-org/gitlab/-/issues/432934note1669995479>>

git:

Target Duration	Percentile	Headroom
-----	-----	-----
5000 ms	p99	3760 ms
5000 ms	p95	4280 ms
5000 ms	p90	4430 ms
5000 ms	p50	4900 ms
1000 ms	p99	500 ms
1000 ms	p95	750 ms
1000 ms	p90	800 ms
1000 ms	p50	900 ms
500 ms	p99	280 ms
500 ms	p95	370 ms
500 ms	p90	400 ms
500 ms	p50	430 ms
250 ms	p99	200 ms
250 ms	p95	230 ms
250 ms	p90	240 ms
250 ms	p50	240 ms

Analysis was done in <<https://gitlab.com/gitlab-org/gitlab/-/issues/432934note1671385680>>

Non-Goals

Not yet defined.

Proposal

The Routing Service implements the following design guidelines:

1. Simple:

- Routing service does not buffer requests.
 - Routing service can only proxy to a single Cell based on request headers.
1. Stateless:
 - Routing service does not have permanent storage.
 - Routing service uses multi-level cache: in-memory, external shared cache.
 1. Zero-trust:
 - Routing service signs each request that is being proxied.
 - The trust is established by using JWT token, or mutual authentication scheme.
 - Cells can be available over public internet, as long as they follow the zero-trust model.
 1. Configuration-based:
 - Routing service is configured with a static list of Cells.
 - Routing service configuration is applied as part of service deployment.
 1. Rule-based:
 - Routing rules are a static JSON file that is part of routing service.
 - Configured rules needs to be made compatible with all versions of GitLab running in a cluster.
 - Rules allows to match by any criteria: header, content of the header, or route path.
 1. Agnostic:
 - Routing service is not aware of high-level concepts like organizations.
 - The classification is done per-specification provided in a rules, to find the classification key.
 - The classification key result is cached.
 - The single classification key cached is used to handle many similar requests.

The following diagram shows how a user request routes through DNS to the Routing Service deployed as Cloudflare Worker and the router chooses a cell to send the request to.

```

mermaid
graph TD;
    user((User));
    router[Routing Service];
    cellus0{Cell US0};
    cellus1{Cell US1};
    celleu0{Cell EU0};
    celleu1{Cell EU1};
    user-->router;
    router-->celleu0;
    router-->celleu1;
    router-->cellus0;
    router-->cellus1;
    subgraph Europe
        celleu0;
        celleu1;
    end
    subgraph United_States
        cellus0;
        cellus1;
    end
  
```

Routing rules

- The routing rules describe how to decode the request, find the classification key, and make the routing decision.
- The routing rules are static and defined ahead of time as part of HTTP Router deployment.
- The routing rules are defined as a JSON document describing in-order a sequence of operation.
- The routing rules might be compiled to application code to provide a way faster execution scheme.
- Each routing rule is described by the cookies, headers, path, method, and action.
- The action can be classify as a way to indicate that the Topology Service should be used to perform dynamic classification.
- The action can be proxy as a way to indicate to perform passthrough to the fixed host that is stored in HTTP Router configuration unless proxy address is specified. Usually, it would be Cell 1 in a cluster.

The routing rules JSON structure describes all matchers:

```
json
{
  "rules": [
    {
      "cookies": {
        "<cookieName>": {
          "matchregex": "<regexmatch>"
        },
        "<cookieName2>": {
          "matchregex": "<regexmatch>"
        }
      },
      "headers": {
        "<headerName>": {
          "matchregex": "<regexmatch>"
        },
        "<headerName2>": {
          "matchregex": "<regexmatch>"
        }
      },
      "path": {
        "matchregex": "<regexmatch>"
      },
      "method": ["<listofacceptedmethods>"],

      "action": "classify",
      "classify": {
        "type": "sessionprefix|projectpath|...",
        "value": "stringbuildfromregexmatchers"
      },

      "action": "proxy",
```

```

    "proxy": {
      "address": "cell1.gitlab.com"
    }
  }
]
}

```

Example of the routing rules that makes routing decision based session cookie, and secret:

```

json
{
  "rules": [
    {
      "cookies": {
        "gitlabsession": {
          "matchregex": "^(?<cellname>cell.*)" // accept gitlabsession that are
prefixed with cell1:
        }
      },
      "action": "classify",
      "classify": {
        "type": "sessionprefix",
        "value": "${cellname}"
      }
    },
    {
      "headers": {
        "GITLABTOKEN": {
          "matchregex": "^(?<cellname>cell.*)" // accept gitlabsession that are
prefixed with cell1:
        }
      },
      "action": "classify",
      "classify": {
        "type": "tokenprefix",
        "value": "${cellname}"
      }
    }
  ]
}

```

Example of the routing rules published by all Cells that makes routing decision based on the path:

```

json
{
  "rules": [
    {

```

```

    "path": {
      "matchregex": "^/api/v4/projects/(?<projectidorpathencoded>[^/]+)(/.)?$"
    },
    "action": "classify",
    "classify": {
      "type": "projectidorpath",
      "value": "${projectidorpathencoded}"
    }
  }
]
}

```

Classification

The classification is implemented by the Classify Service of the Topology Service.

- The classification endpoint uses REST (with mTLS) to secure access.
- The classification endpoint returns only cell name to which information should be routed.
- The classification could return other equivalent classification keys to pollute cache for similar requests.

This is to ensure that all similar requests can be handled quickly without having to classify each time.

- The HTTP Router retries the classify call for a reasonable amount of time.
- The classification for a given value is cached regardless of returned response (positive or negative).

The rejected classification is cached to prevent excessive amount of requests for classification keys that are not found.

- The response does contain Cache- headers that control how long the requests are cached: Cloudflare Workers - Cache.
- If cache is used, the Cache-Tag: is required to be used as a way to have mechanism to selectively wipe a particular type of cache on edge.
- The cache is controlled by Topology Service, but the HTTP Router might force some response into the cache.

For the above example:

1. The router sees request to /api/v4/projects/1000/issues.
1. It selects the above rule for this request, which requests classify for projectidorpathencoded.
1. It decodes projectidorpathencoded to be 1000.
1. Checks the cache if there's projectidorpathencoded=1000 associated to any Cell.
1. Sends the request to /api/v1/classify (type=projectidorpath, value=1000) if no Cells was found in cache.
1. Topology Service responds with the Cell holding the given project, and also all other equivalent classification keys for the resource that should be put in the cache.
1. Routing Service caches for the duration specified in configuration, or response.

```

json
POST /api/v1/classify
Request:
{
  "type": "projectidorpath",
  "value": 1000
}

Response:
{
  "action": "proxy",
  "proxy": {
    "address": "cell1.gitlab.com"
  },
  "otherclassifications": [ // list of all equivalent keys that should be put in the cache
    { "type": "sessionprefix", "value": "cell1" },
    { "type": "projectfullpath", "value": "gitlab-org/gitlab" },
    { "type": "projectfullpath", "value": "gitlab-org/gitlab" },
    { "type": "namespacefullpath", "value": "gitlab-org" }
  ]
}

```

The following code represents a negative response when a classification key was not found:

```

json
POST /api/v4/internal/cells/classify
Request:
{
  "type": "projectidorpath",
  "value": 1000
}

Response:
{
  "action": "reject",
  "reject": {
    "httpstatus": 404
  }
}

```

Configuration

All configuration will be provided via environment variables:

- HTTP Router will only configure an address to Topology Service
- The mTLS will be used when connecting to Topology Service to authentication / authorization.

Deployment

There are several phases to fully deploy the HTTP Routing service to GitLab.com.

1. The first phase is to deploy a simple pass-through proxy in front of the webservice (gitlab.com).

1. First, we will utilize Cloudflare Routes

to rollout the worker gradually, without the need to change DNS.

1. (Maybe optional) The next step is to provision an internal-only DNS for the legacy cell (e.g. cell-1.gprd.int.gitlab.com).

We then proxy the HTTP router to this new DNS, and secure this connection with a solution like mTLS, or Cloudflare Tunnel.

In order to do this, the HTTP Router will need to be assigned the gitlab.com DNS record, likely with custom domains.

1. The second phase is to deploy a simple pass-through proxy in front of the container registry (registry.gitlab.com).

This will use the same deployment of the HTTP Router for gitlab.com.

1. First, we will utilize Cloudflare Routes

to rollout the worker gradually, without the need to change DNS.

1. (Maybe optional) The next step is to provision an internal-only DNS for the legacy cell (e.g. cell-1-registry.gprd.int.gitlab.com).

We then proxy the HTTP router to this DNS, and secure this connection with a solution like mTLS, or Cloudflare Tunnel.

In order to do this, the HTTP Router will need to be assigned the registry.gitlab.com DNS record, likely with custom domains.

1. The third phase involves multiple cells.

1. For any new cell the HTTP Router routes to, the cell will have:

1. An internal-only DNS, like cell-2.gprd.int.gitlab.com that is only accessible via the HTTP Router.

1. A secure, encrypted connection between the HTTP Router and the cell.

Rolling Out Rule Sets

HTTP Router rule sets define the logic how HTTP requests are routed within the Cells environment.

Modifying these rule sets can potentially impact the availability of the entire site or the SLO of any specific service.

Therefore, it is crucial to exercise extreme caution when rolling out changes to the rule sets.

To implement these changes with minimal user impact and zero downtime, we will use the Gradual deployments functionality provided by Cloudflare. This approach allows us to limit the impact of any faulty changes to a small subset of total requests.

The Rule Set that is being used, is configured in the HTTP Router configuration file, where GITLABRULESCONFIG environment variable defines the name of the rule set file relative to src/rules directory.

We will use the existing deployment mechanism. We will gradually increase the rollout percentage, proceeding only when we are confident in the quality and expected outcomes of the rule set changes. The following sequence of rollout percentages is recommended: 5% · 25% · 50% · 75% · 100%.

Prerequisites

- Before processing with rollout steps, make sure you clearly defined the timeline.
- Schedule the change
- Add a new Change Lock entry to the configuration file. Use the http-router Change Lock tag for this entry.

Note: It is important for this rollout strategy to follow the timeline. You will need to merge MRs with a certain interval. Therefore, it's recommended to work in pairs.

Rollout steps

1. Create MR to modify CI configuration of HTTP Router Deployer `.gitlab-ci.yml`. In the global variables section, set both `CHANGELOCKOVERRIDE` and `OVERRIDE LASTPERCENTAGE` environment variables to true linking to a change management issue.
1. In the same MR, change `ROLLOUTPERCENTAGES` environment variable in `deploy-worker.sh` script. Set the value to 5. Example: `ROLLOUTPERCENTAGES="5"`
1. Merge MR.
1. Create and merge MR to update the `GITLABRULESCONFIG` setting inside of `wrangler.toml` to the new rule set.
1. Do any validation for the new rule set and validate that no SLO was effected.
1. Before increasing the `ROLLOUTPERCENTAGES` have some baking time, which can change depending on the environment.
1. If no anomalies found and there is not impact on SLO's repeat step 1 for 25, 50, 75, 100 percents. Keep `CHANGELOCKOVERRIDE` and `OVERRIDE LASTPERCENTAGE` set to true through entire rollout cycle.
1. Once 100% of traffic is rollout out, open MR on `deploy-worker.sh` script to set the value back to the full sequence "5 25 50 75 100". Example: `ROLLOUTPERCENTAGES="5 25 50 75 100"`. Remove the `OVERRIDE LASTPERCENTAGE` and `CHANGELOCKOVERRIDE` environment variables in `.gitlab-ci.yml`.

Request flows

1. There are two Cells.
1. `gitlab-org` is a top-level namespace and lives in Cell US0 in the GitLab.com Public organization.
1. `my-company` is a top-level namespace and lives in Cell EU0 in the my-organization organization.

Router configured to perform the following routing

1. The Cell US0 supports all other public-facing projects.
1. The Cell EU0 configured to generate all secrets and session cookies with a prefix like `celleu0`.
 1. The Personal Access Token is scoped to Organization, and because the Organization is part only of a single Cell, the PATs generated are prefixed with Cell identifier.
 1. The Session Cookie encodes Organization in-use, and because the Organization is part only of a single Cell, the session cookie generated is prefixed with Cell identifier.
1. The Cell EU0 allows only private organizations, groups, and projects.

1. The Cell US0 is a target Cell for all requests unless explicitly prefixed.

Router rules:

```
json
{
  "rules": [
    {
      "cookies": {
        "gitlabsession": {
          "regexmatch": "^(?<cellname>cell.*)"
        }
      },
      "action": "classify",
      "classify": {
        "type": "sessionprefix",
        "value": "${cellname}"
      }
    },
    {
      "headers": {
        "GITLABTOKEN": {
          "regexmatch": "^(?<cellname>cell.-)"
        }
      },
      "action": "classify",
      "classify": {
        "type": "tokenprefix",
        "value": "${cellname}"
      }
    },
    {
      "action": "classify",
      "classify": {
        "type": "firstcell",
      }
    }
  ]
}
```

Goes to /my-company/my-project while logged in into Cell EU0

1. Because user switched the Organization to my-company, its session cookie is prefixed with celleu0.
1. User sends request /my-company/my-project, and because the cookie is prefixed with celleu0 it is directed to Cell EU0.
1. Cell EU0 returns the correct response.

mermaid

sequenceDiagram

```
participant user as User
participant router as Router
participant cache as Cache
participant ts as Topology Service
participant celleu0 as Cell EU0
participant celleu1 as Cell EU1
user->>router: GET /my-company/my-project<br/>gitlabsession=celleu0uwwz7rdavi9
router->>+cache: GetClassify(type=sessionprefix, value=celleu0)
cache->>-router: NotFound
router->>+ts: Classify(type=sessionprefix, value=celleu0)
ts->>-router: Proxy(address="cell-eu0.gitlab.com")
router->>cache: Cache(type=sessionprefix, value=celleu0) = Proxy(address="cell-
eu0.gitlab.com"))
router->>celleu0: GET /my-company/my-project
celleu0->>user: <h1>My Project...
```

Goes to /my-company/my-project while not logged in

1. User visits /my-company/my-project, and because it does not have session cookie, the request is forwarded to Cell US0.
1. User signs in.
1. GitLab sees that user default organization is my-company, so it assigns session cookie with celleu0 to indicate that user is meant to interact with my-company.
1. User sends request to /my-company/my-project again, now with the session cookie that proxies to Cell EU0.
1. Cell EU0 returns the correct response.

NOTE:

The cache is intentionally skipped here to reduce diagram complexity.

mermaid

sequenceDiagram

```
participant user as User
participant router as Router
participant ts as Topology Service
participant cellus0 as Cell US0
participant celleu0 as Cell EU0
user->>router: GET /my-company/my-project
router->>ts: Classify(type=firstcell)
ts->>router: Proxy(address="cell-us0.gitlab.com")
router->>cellus0: GET /my-company/my-project
cellus0->>user: HTTP 302 /users/signin?redirect=/my-company/my-project
user->>router: GET /users/signin?redirect=/my-company/my-project
router->>cellus0: GET /users/signin?redirect=/my-company/my-project
cellus0-->>user: <h1>Sign in...
user->>router: POST /users/signin?redirect=/my-company/my-project
router->>cellus0: POST /users/signin?redirect=/my-company/my-project
```

```

cellu0->>user: HTTP 302 /my-company/my-project<br/>gitlabsession=celleu0uwwz7rdavi9
user->>router: GET /my-company/my-project<br/>gitlabsession=celleu0uwwz7rdavi9
router->>ts: Classify(type=sessionprefix, value=celleu0)
ts->>router: Proxy(address="cell-eu0.gitlab.com")
router->>celleu0: GET /my-company/my-project<br/>gitlabsession=celleu0uwwz7rdavi9
celleu0->>user: <h1>My Project...

```

Goes to /gitlab-org/gitlab after last step

User visits /gitlab-org/gitlab, and because it does have a session cookie, the request is forwarded to Cell EU0.

There is no need to ask Topology Service, since the session cookie is cached.

mermaid

sequenceDiagram

```

    participant user as User
    participant router as Router
    participant cache as Cache
    participant ts as Topology Service
    participant celleu0 as Cell EU0
    participant celleu1 as Cell EU1
    user->>router: GET /my-company/my-project<br/>gitlabsession=celleu0uwwz7rdavi9
    router->>+cache: GetClassify(type=sessionprefix, value=celleu0)
    cache->>-router: Proxy(address="cell-eu0.gitlab.com")
    router->>celleu0: GET /my-company/my-project
    celleu0->>user: <h1>My Project...

```

Performance and reliability considerations

- It is expected that there will be penalty when learning new classification key. However, it is expected that multi-layer cache should provide a very high cache-hit-ratio, due to low cardinality of classification key. The classification key would effectively be mapped into resource (organization, group, or project), and there's a finite amount of those.

Alternatives

Buffering requests

The Stateless Router using Requests Buffering

describes an approach where Cell answers with X-Gitlab-Cell-Redirect to redirect request to another Cell:

- This is based on a need to buffer the whole request (headers + body) which is very memory intensive.
- This proposal does not provide an easy way to handle mixed deployment of Cells, where Cells might be running different versions.
- This proposal likely requires caching significantly more information, since it is based on

requests, rather than on decoded classification keys.

Learn request

The Stateless Router using Routes Learning describes an approach similar to the one in this document. Except the route rules and classification is done in a single go in a form of pre-flight check `/api/v4/internal/cells/learn`:

- This makes the whole routes learning dynamic, and dependent on availability of the Cells.
- This proposal does not provide an easy way to handle mixed deployment of Cells, where Cells might be running different versions.
- This proposal likely requires caching significantly more information, since it is based on requests, rather than on decoded classification keys.

Single Domain

To maintain a single domain for all cells, the webserver needs to respond as the public host when performing redirects. The BYOD feature for dedicated serves this purpose by allowing the cell to behave as though it's serving on the public domain.

Example byod config snippet. Note - Only the instance is configured, not kas or registry domain

```
json
"byod": {
  "instance": "gitlab.com",
}
```

Domain Setup

- Production cells configured using BYOD public domain (eg., gitlab.com)
- Each Cell also responds to their configured manageddomain
- Nginx ingress handles both domains

SSL/TLS Configuration

- The configured BYOD domain should have the certificate managed already, and it's not something instrumentor is managing.
- The manageddomain is handled using cert-manager DNS solver as the default http solver won't work behind a proxy particularly when the cells will not be publically routable

Nginx-ingress Configuration

- Listens on both the primary domain and the manageddomain domains
- Processes the X-Forwarded-Host header for proper routing, the host header can't be used so nginx uses X-Forwarded-Host which is passed by the router

Cell infrastructure routing

Uses the sessionprefix rule described here

mermaid

sequenceDiagram

participant User as User (Browser)

participant HTTPRouter as HTTP Router (Cloudflare)

participant TopologyService as Topology Service

box Cell

participant CellIngress as Cell Ingress (nginx-ingress)

participant CellWebservice as Webservice Container(workhorse/puma)

end

participant LegacyCell as Legacy Cell

Note over User,CellWebservice: Cell-based routing path

User->>HTTPRouter: Cookie: gitlabsession=cell-\$ID-xxx

HTTPRouter->>+TopologyService: Query Cell ID extract from gitlabsession

TopologyService-->>-HTTPRouter: Return manageddomain for Cell

HTTPRouter->>HTTPRouter: Set X-Forwarded-Host: gitlab.com

HTTPRouter->>CellIngress: Proxy to manageddomain

CellIngress->>CellIngress: Use Host header matching X-Forwarded-Host

CellIngress->>+CellWebservice: Proxy with Host: gitlab.com

CellWebservice-->>-CellIngress: Response

CellIngress-->>HTTPRouter: Response

HTTPRouter-->>User: Response

Note over User,LegacyCell: Default routing (no cell prefix in gitlabsession)

User->>HTTPRouter: Cookie: gitlabsession=xxxx

HTTPRouter->>LegacyCell: Proxy to GitLab.com

LegacyCell-->>HTTPRouter: Response

HTTPRouter-->>User: Response

FAQ

1. How and when will Routing Service compile set of rules?

To be defined.

Links

- Cells - Routing: Technology
- Classify endpoints

SECTION: engineering/architecture/design-documents/cells/migration.md

Summary

When we reach production and can operate organizations on new cells, we need to be able to move organizations from GitLab.com into a cell or between any two cells. The Org Mover project provides a CLI orchestrator tool that makes the process of moving organization data from one cell to another a breeze. It's designed to ensure no data loss or corruption with minimal downtime.

It is important to note that the tool itself isn't responsible for moving data, but it takes care of setting up all the required logistics. It will be developed as a gem within the `gitlab-org/gitlab` codebase.

Motivation

Org Mover will allow organizations to be moved to our scalable cells architecture, future-proofing our GitLab.com services as GitLab grows. This way, we guarantee that we will be able to deliver a consistently excellent service for our GitLab.com customers with all the advantages that come with the cells architecture.

Goals

- Move organizations off GitLab.com or between Cells to rebalance them with minimal downtime:
 - Maximum downtime requirements for moving an organization and cutting it over to the target cell are as follows:
 - Cells 1.5 - 24 hours
 - Cells 2.0 - Ideally 0. Low enough to not have to coordinate the migration with them.
 - Reuse Geo code to replicate non-PostgreSQL data continuously for a particular organization.
 - Implement a reliable and scalable tool to copy all PostgreSQL data for a particular organization from the source database and writes it to the target database present in another cell.
- Create the Org mover CLI orchestrator tool that will:
 - Set up and manage the replication of non-PostgreSQL data for a particular organization.
 - Put an organization into a maintenance mode.
 - Start the PostgreSQL data copy.
 - Handle tables exempt from sharding.
 - Handle the Elasticsearch and Zoekt data.
 - Rebuild the ClickHouse data.
 - Monitor the replication process.
 - Perform the switchover.
 - Remove all organization data that has been moved over.
- Implement robust error handling and monitoring to quickly identify and resolve any issues in the data synchronization process.
- Ensure there's no data loss or corruption during the move. The process must ensure that all data at the source cell is copied to the target cell.
- Ensure the process for moving data from one cell to another is timely and allows for reasonable throughput. Moving many organizations simultaneously shouldn't put undue strain on the infrastructure.
- Ensure there is no downtime while turning on/off the replication on live systems. Both GitLab.com and the target cell will have live organizations with